
Tool analysis for X-CUBE-STL functional safety support package

Introduction

This document applies to the **X-CUBE-STL** Expansion Package. It includes the results of the tool analysis executed in accordance with the requirements of IEC 61508-3 section 7.4.4 to specific support tools that may be used in the framework of the X-CUBE-STL integration in a final application.

This document is written in compliance with the IEC 61508 international norm for functional safety of electrical/electronic/programmable electronic safety-related systems. The version used as a reference is IEC 61508-1/7:2010.

ST Restricted





1 About this document

The X-CUBE-STL functional safety design package runs on STM32 32-bit microcontrollers based on the Arm[®] Cortex[®]-M processor.

Note: Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

1.1 Purpose and scope

The purpose of this document is to collect the results of the tool analysis executed in accordance with the requirements of IEC 61508-3 section 7.4.4 to specific support tools that may be used in the framework of the X-CUBE-STL integration in an end user final application.

Note: The analysis methodology followed may differ from one tool to another, as the best pattern among the ones proposed by the safety standard is chosen.

1.2 Terms and abbreviations

Table 1. Terms and abbreviations

Acronym	Definition
CLI	Command-line interface
CRC	Cyclic redundancy check
MCU	Microcontroller unit
MM	Mitigation measure
MPU	Microprocessing unit
UC	Use case

1.3 Normative reference

This document is written in compliance with the IEC 61508 international norm for functional safety of electrical/electronic/programmable electronic safety-related systems.

The version used as a reference is IEC 61508-1/7:2010.



2 CRC tool analysis

2.1 Tool presentation

2.1.1 Scope

This section includes the tool analysis for the CRC precalculation tool, available as a single function inside the more generic tool STM32CubeProgrammer (STM32CubeProg). Until the publication of this document, the tool is provided as an example of how to perform the correct CRC calculation in the framework of the requirements/needs of the flash memory test performed by the software diagnostic X-CUBE-STL. As the tool is supposed to modify the object code (by adding CRC seeds at the end of the memory), it must be classified T3 when directly used. As such, it falls under IEC 61508-3 related requirements.

Because the intended function (CRC precalculation) is offered as a single function inside a wider software tool, the fulfillment of the IEC 61508-3 7.4.4.6 clause about the provision of evidences related to tool validation is not pursued in this document. Instead, as the tool function is quite simple and the intended use is well defined, this section targets the possibility opened by IEC 61508-3 7.4.4.8, by collecting all needed analysis, to provide to STMicroelectronics customers clear instructions on how to mitigate potential tool failures, with measures on their side like additional checks on tool output, functionality, restricted use, and others. It is worth to note that the selected pattern (clause 7.4.4.8) is reasonably possible given the outcomes of the tool analysis detailed in the following sections, for which an effective mitigation of potential tool failure is possible by relatively simple means.

Remember: *The end user remains fully responsible of all mitigation actions if the tool is used in the framework of an IEC 61508-compliant development.*

2.1.2 Tool name and release

This tool is a single function:

Safety lib command (-sl, --safelib)

It is available in the STM32CubeProgrammer (STM32CubeProg) command-line interface (CLI) for MCUs and MPUs.

2.1.3 Tool version and related compatibility

- STM32CubeProgrammer (STM32CubeProg) v2.10.0

2.1.4 Tool documentation

- *STM32CubeProgrammer software description* (UM2237)
This is the main reference documentation for the end user. It lists all the tool specific features (refer to “Safety lib command”).
- *STM32xx Series self-test library user guide*
This is a generic reference: Any available user guide for an STM32 MCU series can be used as source, as the contents are fully shared among the series.
This document is the user guide of X-CUBE-STL. In the section “Flash memory tests”, it provides information on the possible use of the CRC tool and its settings.

2.1.5 General description

The CRC precalculation tool is available as a single function inside the more generic tool STM32CubeProgrammer (STM32CubeProg). It is invoked by its dedicated CLI command. It modifies a firmware file, provided as input, to add in a specific address area (segment) a certain number of CRC seeds computed over the preexisting firmware memory image. The scope of the tool is to fill the dedicated area automatically with the CRC seeds to be used by the software diagnostics X-CUBE-STL for the flash memory integrity test. The supported formats are .bin, .elf, .hex, and .srec. The command is configurable in order to adapt tool behavior to both:

- the memory start address and end address
- the needed size for the data block on which the tool computes the CRC

The X-CUBE-STL documentation (user guides) provides the correct settings to be used in the CLI tool invocation. The tool modifies the firmware image also to add, if needed, trailing zeros to align to 32-bit; this feature as well is needed for the compatibility with X-CUBE-STL flash memory tests.



2.2 Use cases

2.2.1 UC1: CRC tool used to modify directly the final object file

2.2.1.1 General description

The CRC tool is used to modify directly the application software firmware image by adding the required CRC seeds. The firmware image obtained is used as it is for further phases on the final product development.

Note: This use case assumes that the files manipulations are done by a human operator and not by automated scripting. Visual inspection by the human operator is part of the use case.

2.2.1.2 Inputs

The tool is fed by the following information during its CLI invocation:

- Input file name
- Start and stop memory addresses
- Size of data per CRC value

The above data is assumed to be correctly fed into the tool (besides any tool malfunctions). It is assumed to reflect properly the nature of the intended end user application.

Important: This use case foresees the object code only as a valid output format of the CRC tool.

2.2.1.3 Execution

The tool computes the required CRC values over the specified memory space, by using the requested data packets length. The values are used then for firmware modification.

Therefore, two operations modify the firmware memory image:

- The CRC data is added to the dedicated area (top of the memory space)
- If the original firmware image is not to 32-bit aligned, trailing zeros (0x00) are added to the end of the image to perform the alignment.

No other modification is done to the binary object code image.

2.2.1.4 Output

The input file (specified in the above [Inputs](#) phase) is overwritten with the object code image obtained during the above [Execution](#) phase. The object code file is used as it is by the following end user phases (such as the end product programming).

2.3 Tool analysis

This section includes the tool analysis by leveraging on the methods proposed in IEC 61508-3 section 7.4.4.

2.3.1 Tool analysis

The table below details the complete failure analysis and the related proposed mitigation measures for the CRC tool.



The table columns provide the following information:

- ID#: unique identifier for each failure.
- Function description: the specific tool function for which the failure is analyzed.
- Tool failure: the description of the function failure potentially leading to errors in the executable code.
- Tool classification: the tool classification according to IEC 61508-4 section 3.2.11 in dependency to the specific failure analyzed. Therefore, the classification may differ from one failure to the other.
Note that the presence of T3 failures leads automatically to rank the tool as T3
- Mitigation measure type: a generic classification for the mitigation measure, as proposed in IEC 61508-3 7.4.4.5 Note 2.
- Mitigation measure description: high-level description of the mitigation measure. Refer to [Section 2.3.2](#) for the detailed explanation.
- Note: provides an explanation of the effectiveness of the proposed mitigation measure to contain/detect the specific tool failure.
- Failure mitigated: final evaluation of the specific tool failure mitigation (Yes/No).


Table 2. CRC tool analysis

ID#	Function description	Tool failure	Tool classification	Mitigation measure type	Mitigation measure description	Note	Failure mitigated
#1	CRC insertion in object file	One or multiple CRC seeds computed with a wrong value.	T3	Tool output check	MM1: X-CUBE-STL integration tests	During the integration tests, the X-CUBE-STL finds a mismatch between the expected CRC seeds values (contained in the X-CUBE-STL object code) and the ones stored in the flash memory, so that a FAIL error is produced.	Yes
#2	CRC insertion in object file	One or multiple CRC seeds computed with memory slices size different from what is specified in the command line.	T3	Tool output check	MM1: X-CUBE-STL integration tests	The CRC seeds stored in the flash memory differ from the expected ones. The X-CUBE-STL integration test acts as in #1.	Yes
#3	CRC insertion in object file	Missing the insertion of one or multiple CRC seeds in the load area.	T3	Tool output check	MM1: X-CUBE-STL integration tests	As the original values of the load area are different from the expected CRC seeds values missed to be inserted, the integration test MM1 is able to detect the mismatch as in #1.	Yes
#4	CRC insertion in object file	Load area (segment containing the CRC seeds) placed at the wrong memory address (at a free area).	T3	Tool output check	MM1: X-CUBE-STL integration tests	During the integration tests, the X-CUBE-STL does not find the expected CRC seeds values at the planned memory address. A FAIL error is produced.	Yes
#5	CRC insertion in object file	Load area (segment containing the CRC seeds) placed at the wrong memory address (overlap with the original application software object image).	T3	Tool output check	MM1: X-CUBE-STL integration tests	This failure results in both an original object code mutation and a wrong place for the CRC seed. It is covered by the failures analyzed in #6 and #4.	Yes
#6	Object code manipulation	Original application software firmware image copied to the output firmware with errors in data.	T3	Tool output check	MM2: post-CRC insertion comparison for object files	The comparison between the original and the final object files reveals errors in the copy operation.	Yes
#7	Object code manipulation	Original application software firmware image copied to the output firmware at wrong addresses.	T3	Tool output check	MM2: post-CRC insertion comparison for object files	The comparison between the original and the final object files reveals errors in the copy operation	Yes
#8	Trailing zeros insertion	Missing the insertion of one or multiple trailing zeros in the object output file.	T3	Tool output check	MM1: X-CUBE-STL integration tests	The CRC seeds are supposed to be computed with trailing zeros where needed. The X-CUBE-STL execution raises a FAIL error during the integration test since the computed CRC is different.	Yes
#9	Trailing zeros insertion	Insertion of non-zero values at address locations that were the targets for the insertion of trailing zeros.	T3	Tool output check	MM1: X-CUBE-STL integration tests	Same as #8.	Yes
#10	Trailing zeros insertion	Insertion of trailing zeros at locations occupied by the original application software firmware.	T3	Tool output check	MM2: post-CRC insertion comparison for object files	The comparison between the original and the final object files reveals errors in the copy operation	Yes



ID#	Function description	Tool failure	Tool classification	Mitigation measure type	Mitigation measure description	Note	Failure mitigated
#11	Command line invocation and parameter translation	Error in the command line parsing, leading to wrong input file processing (or no file processing at all).	T3	Tool output check	MM2: post-CRC insertion comparison for object files	The comparison clearly fails.	Yes
#12	Command line invocation and parameter translation	Error in the command line parsing, leading to a wrong start address, wrong end address, or both.	T3	Tool output check	MM1: X-CUBE-STL integration tests	Wrong and/or incomplete computation of CRC seeds is detected by the integration test.	Yes
#13	Command line invocation and parameter translation	Error in the command line parsing, leading to a wrong memory slice.	T3	Tool output check	MM1: X-CUBE-STL integration tests	Failure equivalent to the one analyzed in #2.	Yes
#14	Command line invocation and parameter translation	Tool malfunction leading to no object file modification at all (input translated directly to the output, output file saved with a different name, in a different place, or both, or no operation performed at all).	T3	Tool output check	MM1: X-CUBE-STL integration tests MM2: post-CRC insertion comparison for object files	The use case does not foresee the automatic use of the CRC tool output, but instead requires multiple verification activities performed by the end user (the one described in this analysis). This kind of failure is detected by multiple verification items.	Yes
#15	ELF file manipulation	Any tool failure leading to errors in the production of the output modified ELF file.	T1	Functionality restricted use	Not applicable	The ELF file used by the end user is excluded by this tool use case. The related functionality is not able to interfere with the safety functions, therefore the related failures are classified as T1.	Yes



2.3.2 Mitigation measures

This section includes the list of mitigation measures proposed in the [Section 2.3.1 Tool analysis](#) above.

2.3.2.1 MM1: X-CUBE-STL integration tests

The integration tests are prescribed in X-CUBE-STL as part of the verification activity needed for the use of the STL itself in a safety application. Among these tests, the execution of the flash memory test function is required. During those tests, the CRC seeds programmed in the memory are accessed by the X-CUBE-STL and compared with the CRC computed by hardware means (STM32 CRC internal module) or software means (CRC software function available in X-CUBE-STL) on the actual flash memory contents. Any mismatch between the expected and the computed CRC seeds is reported as a FAIL error and therefore clearly diagnosed during the integration tests; fails of the verification may be caused (among other different causes) by a CRC tool failure.

Note: The robustness of this mitigation measure is augmented by the diversity of the CRC computation. Indeed, the CRC computation in the STM32 hardware module and in the X-CUBE-STL software are both diverse implementations from the CRC computation engine in the tool under assessment. Therefore, the diversity is guaranteed, regardless of the type of CRC scheme adopted in X-CUBE-STL (hardware or software) during the integration tests. Furthermore, the above verification procedure leverages on the checks performed by functions of X-CUBE-STL, which is certified by TÜV Rheinland (see the certificates 968/FSP 1627.00/18 and 968/FSP 2292.00/21) for systematic capability (SC3) compliance with IEC 61508-3.

2.3.2.2 MM2: post-CRC insertion comparison for object files

The binary object file obtained by the execution of the CRC seeds injection by the CRC tool is compared with the original (not modified) object file. The commercial tools usually available for file comparison can do this verification. The scope of the verification is to check that:

- the original object code image has been correctly copied into the CRC-modified object code, with no changes in the values or in the address
- the CRC seeds (load area - segment) are placed in the expected memory zone, as per the tool and the X-CUBE-STL user guide specifications

Any mismatch must be considered as caused by a CRC tool failure.

Note: As this verification requires the use of a second tool (file comparison tool), which could be in principle considered classified as T2, the related issues from a tool verification point of view can be easily solved by performing the above specified comparison with two different commercial tools (IEC 61508-3 7.4.4.5 Note 2: "use of diverse tools for the same purpose.").

2.3.3 Conclusions

The outcomes of the tool analysis performed in the [Table 2](#) above and the inspection of the proposed mitigation measures bring to the following:

- the proposed measures effectively mitigate all potential tool failures
- The proposed measures can be implemented on the end user side with a reasonable amount of effort and complexity

It is possible therefore to conclude that the whole tool analysis included in this [Section 2](#) enables the end user to follow the pattern allowed by the clause IEC 61508-3 7.4.4.8 for the use of the CRC tool in a safety-compliant development model.

It is reminded that:

- The end user is fully responsible for the correct implementation of the proposed mitigation measures, and for all related verification activities
- It is assumed that the whole argument related to the tool usage is submitted to an assessment/evaluation during the certification process of the end user final application



Revision history

Table 3. Document revision history

Date	Revision	Changes
27-Jun-2022	1	Initial release.

ST Restricted



Contents

1	About this document	2
1.1	Purpose and scope	2
1.2	Terms and abbreviations	2
1.3	Normative reference	2
2	CRC tool analysis	3
2.1	Tool presentation	3
2.1.1	Scope	3
2.1.2	Tool name and release	3
2.1.3	Tool version and related compatibility	3
2.1.4	Tool documentation	3
2.1.5	General description	3
2.2	Use cases	4
2.2.1	UC1: CRC tool used to modify directly the final object file	4
2.3	Tool analysis	4
2.3.1	Tool analysis	4
2.3.2	Mitigation measures	8
2.3.3	Conclusions	8
	Revision history	9
	List of tables	11



List of tables

Table 1.	Terms and abbreviations	2
Table 2.	CRC tool analysis	6
Table 3.	Document revision history	9

ST Restricted

**IMPORTANT NOTICE – READ CAREFULLY**

STMicroelectronics International NV and its affiliates (“ST”) reserve the right to make changes corrections, enhancements, modifications, and improvements to ST products and/or to this document any time without notice.

This document is provided solely for the purpose of obtaining general information relating to an ST product. Accordingly, you hereby agree to make use of this document solely for the purpose of obtaining general information relating to the ST product. You further acknowledge and agree that this document may not be used in or in connection with any legal or administrative proceeding in any court, arbitration, agency, commission or other tribunal or in connection with any action, cause of action, litigation, claim, allegation, demand or dispute of any kind. You further acknowledge and agree that this document shall not be construed as an admission, acknowledgement or evidence of any kind, including, without limitation, as to the liability, fault or responsibility whatsoever of ST or any of its affiliates, or as to the accuracy or validity of the information contained herein, or concerning any alleged product issue, failure, or defect. ST does not promise that this document is accurate or error free and specifically disclaims all warranties, express or implied, as to the accuracy of the information contained herein. Accordingly, you agree that in no event will ST or its affiliates be liable to you for any direct, indirect, consequential, exemplary, incidental, punitive, or other damages, including lost profits, arising from or relating to your reliance upon or use of this document.

Disclosure of this document to any non-authorized party must be previously authorized by ST only under the provision of a proper confidentiality contractual arrangement executed between ST and you and must be treat as strictly confidential.

At all times you will comply with the following securities rules:

- Do not copy or reproduce all or part of this document.
- Keep this document locked away.
- Further copies can be provided on a “need to know basis”. Contact your local ST Sales Office or document writer.

Purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST’s terms and conditions of sale in place at the time of order acknowledgment, including, without limitation, the warranty provisions thereunder.

In that respect, note that ST products are not designed for use in some specific applications or environments described in above mentioned terms and conditions.

Purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of purchasers’ products.

Information furnished is believed to be accurate and reliable. However, ST assumes no responsibility for the consequences of use of such information nor for any infringement of patents or other rights of third parties which may result from its use. No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2022 STMicroelectronics – All rights reserved